



Smoothing and Discretization



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Discretization and Smoothing

Fundamentals of numerical data preprocessing

In the context of data preprocessing, discretization is the process of transforming continuous functions, models, variables, and equations into their discrete counterparts. This transformation becomes essential when we need to work with algorithms that require categorical data or when we want to reduce the inherent complexity of continuous numerical data.

Closely related to discretization, smoothing complements this process by creating approximation functions that capture the most relevant patterns in the data, while eliminating noise and fine-scale structures. Together, these techniques are crucial for revealing underlying trends in noisy data, making analysis more accurate and interpretable.

A central aspect for the application of these techniques is the definition of intervals, or *bins*, which determine how data will be grouped during approximation. The careful choice of these intervals directly influences the quality of results — inadequate binning can both obscure real patterns and introduce unwanted artifacts into the analysis.

- ❏ **Important references:** Kerber (1992) developed the ChiMerge method for discretization of numerical attributes. Liu et al. (2002) demonstrated that discretization is a fundamental enabling technique in data mining.

Binning Methods for Data Smoothing

With the concepts of discretization and binning established, we can examine the two main strategies for defining intervals — each with its own characteristics that make them more suitable for different types of distributions.

Equal-Width Partitioning

Divides the range into N intervals of equal size, creating a uniform grid. If A and B are the lowest and highest values of the attribute, the width of the intervals will be: $W = (B - A)/N$.

- Most straightforward and simple approach to implement
- Outliers can dominate the data presentation
- Skewed data is not handled adequately
- Works well for uniform distributions

Equal-Depth Partitioning

While equal-width prioritizes spatial uniformity, this approach divides the range into N intervals, each containing approximately the same number of samples — ensuring balanced representativeness across bins.

- Excellent data scalability
- Each bin has similar representativeness
- Managing categorical attributes can be challenging
- More robust for skewed data

❏ Krzywinski (2016) highlights that the choice between equal-width and equal-depth is not trivial: in high-resolution data, the wrong strategy can obscure critical patterns or distort scientific visualizations. In practice, the decision depends directly on the data distribution — a topic we will explore in the practical comparison that follows.

Practical Comparison of Binning Methods

To illustrate the differences between binning methods, consider sorted price data (in dollars): **4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34**. Applying the two methods presented earlier to this same dataset, we can concretely observe how each strategy distributes values into bins.

Equal-Width Partitioning (3 bins)

Calculation: $(34-4)/3 = 10$ units per bin

- **Bin 1 [4-13[**: 4, 8, 9
- **Bin 2 [14-23[**: 15, 21, 21
- **Bin 3 [23-34]**: 24, 25, 26, 28, 29, 34

With the bins defined, the next step is to apply **smoothing** techniques over the grouped values. Using equal-frequency partitioning as a basis, we can reduce noise in the data in two distinct ways: replacing each value with the mean of its bin or with the nearest boundary.

Smoothing Techniques Using Equal Frequency

Smoothing by Bin Means

Each value is replaced by the mean of its bin:

- **Bin 1**: 9, 9, 9, 9 (mean = 9)
- **Bin 2**: 23, 23, 23, 23 (mean = 23)
- **Bin 3**: 29, 29, 29, 29 (mean = 29)

Equal-Frequency Partitioning (3 bins)

4 values per bin (12 values ÷ 3 bins)

- **Bin 1**: 4, 8, 9, 15
- **Bin 2**: 21, 21, 24, 25
- **Bin 3**: 26, 28, 29, 34

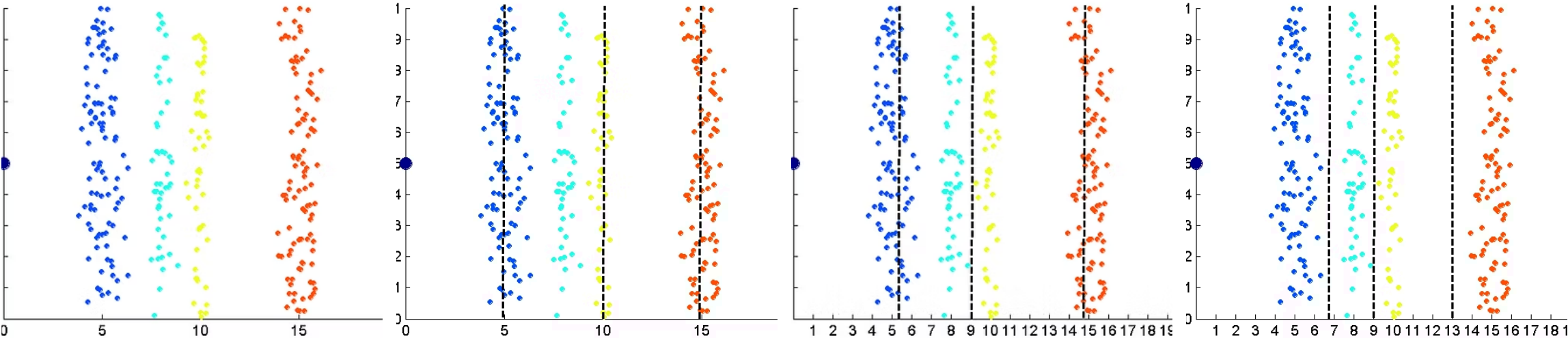
Smoothing by Bin Boundaries

Each value is replaced by the nearest boundary:

- **Bin 1**: 4, 4, 4, 15
- **Bin 2**: 21, 21, 25, 25
- **Bin 3**: 26, 26, 26, 34

Influence of Binning on Discretization Techniques

Having seen how binning methods distribute and smooth data numerically, we can now observe this impact **visually**. The charts below illustrate how the choice of number of bins and partitioning strategy transforms the distribution of the same original data — revealing distinct patterns depending on the approach adopted:



01

Original Data

Starting point: raw distribution with all data points without any processing, highlighting variability and possible noise

02

Equal-Width Binning

From the raw data, smoothing is applied with 4 uniform-width bins — fixed intervals that may generate bins with very different densities

03

Equal-Frequency Binning

An alternative to the previous method, here each bin receives a similar number of points, resulting in a more balanced and representative smoothing

04

K-means Clustering

Going beyond interval-based methods, clustering-based discretization groups points by intrinsic similarity, capturing natural structures in the data

Overview – Smoothing and Discretization

With the visual effects of binning already demonstrated, it's time to examine **how each method is implemented in practice**. The three algorithms below represent distinct approaches to discretizing continuous variables — each will be detailed with code and examples in the upcoming slides:

smoothing_inter

Discretization by equal-width intervals. Divides the domain into N bins of the same size. Simple, but sensitive to outliers.

smoothing_freq

Discretization by equal frequency (quantiles). Each bin contains approximately the same number of instances. More robust for asymmetric distributions.

smoothing_cluster

Clustering-based discretization (k-means). Bins are defined by cluster centroids, capturing the natural structure of the data.

What comes next: For each method above, the upcoming slides present the code implementation — including the main parameters, expected behavior, and output examples. The choice of number of bins directly impacts the quality of the transformation and will be explored in each case.

Equal-Width Interval Discretization – Theory

Equal-width discretization divides the continuous value range $[\min, \max]$ into N bins of identical width $W = (\max - \min) / N$. Each bin covers the same numeric range, regardless of data density, with fixed, evenly spaced boundaries.

Strengths

- Simple & Interpretable
- Easy to implement
- Ideal for uniform distributions

Weaknesses

- Sensitive to outliers (distorts all bins)
- Poor for skewed distributions
- May produce empty bins

- Dougherty et al. (1995) established the theoretical basis for these methods in "Supervised and Unsupervised Discretization of Continuous Features" (ICML).

Interval Discretization

```
library(daltoolbox)

iris <- datasets::iris

# Discretização em 2 intervalos iguais
obj <- smoothing_inter(n = 2)
obj <- fit(obj, iris$Sepal.Length)
sl.bi <- transform(obj, iris$Sepal.Length)
print(table(sl.bi))
# 5.328 6.733
# 95 55
obj$interval # [1] 4.3 6.1 7.9
```

```
# Entropia condicional
entro <- evaluate(obj,
  as.factor(names(sl.bi)), iris$Species)
print(entro$entropy) # [1] 1.191734

# Otimizando número de bins (1:20)
opt_obj <- smoothing_inter(n=1:20)
obj <- fit(opt_obj, iris$Sepal.Length)
obj$n # [1] 8 (ótimo encontrado)
```

Step by step

The first of the three methods presented is the most straightforward: it divides the domain into bins of **equal width**, without considering the data distribution.

- Fitting with `fit()` defines **equal-width** intervals over the variable's range
- With `n=2`: interval `[4.3, 6.1]` and `[6.1, 7.9]` — 95 and 55 instances
- `evaluate()` computes the conditional entropy between bins and classes
- Passing `n=1:20`, the method automatically searches for the optimal number of bins (8)

❏ **Key concept:** Equal-width intervals are simple and interpretable, but can create unbalanced bins for skewed distributions — as seen here (95 vs. 55 instances). To overcome this limitation, the next method, `smoothing_freq`, distributes instances more uniformly across bins using quantiles.

Equal-Frequency Discretization – Theory

Equal-frequency discretization divides the data range into N bins such that each bin contains approximately the same number of observations (n/N instances). Unlike equal-width, the bin cut points are determined by the data's quantiles, allowing bin widths to vary. Bins are narrower where data is dense and wider where data is sparse, adapting to the underlying distribution.

Strengths

- Avoids empty or sparsely populated bins
- Robust to skewed data distributions
- Each bin offers similar statistical representativeness
- Often leads to better class separation (lower entropy)

Weaknesses

- Bin boundaries can be harder to interpret
- May inadvertently split natural clusters
- Can be challenging with categorical attributes

❏ Fayyad & Irani (1993) introduced "Multi-Interval Discretization of Continuous-Valued Attributes for Classification Learning" (IJCAI), providing foundational work for frequency-based and entropy-based methods.

Frequency-Based Discretization

```
library(daltoolbox)

iris <- datasets::iris

# Discretização em 2 bins por frequência
obj <- smoothing_freq(n = 2)
obj <- fit(obj, iris$Sepal.Length)
sl.bi <- transform(obj, iris$Sepal.Length)
print(table(sl.bi))
# 5.199 6.580
# 80 70
obj$interval # [1] 4.3 5.8 7.9
```

```
# Entropia condicional
entro <- evaluate(obj,
  as.factor(names(sl.bi)), iris$Species)
print(entro$entropy) # [1] 1.097573

# Otimizando número de bins (1:20)
opt_obj <- smoothing_freq(n=1:20)
obj <- fit(opt_obj, iris$Sepal.Length)
obj$n # [1] 8
```

Step by step

To resolve the imbalance generated by `smoothing_inter` (95 vs. 55 instances), `smoothing_freq` defines the cut points based on **quantiles**, ensuring that each bin receives approximately the same number of observations.

- Frequency-based bins (quantiles): each bin contains approximately the same number of instances
- With `n=2`: 80 and 70 instances — more balanced than equal-width intervals
- Lower entropy (1.097 vs. 1.192) indicates better class separation
- The optimal number of bins is also 8, the same as the interval-based method

📌 **Key concept:** Frequency-based discretization (quantiles) avoids empty bins and is more robust to skewed distributions than equal-width intervals. However, both `smoothing_inter` and `smoothing_freq` define cut points without considering the **underlying structure of the data** — a limitation that the next method, `smoothing_cluster`, overcomes by using clustering to identify the most natural cut points.

Clustering-Based Discretization – Theory

Clustering-based discretization leverages 1D k-means clustering to identify natural groupings within continuous data. Instead of predefined rules, it dynamically determines bin cut points based on the inherent structure and density of the dataset.

The algorithm iteratively assigns each data point to its nearest cluster centroid and then updates these centroids based on the new assignments. This process continues until the clusters stabilize. The bin boundaries are then set at the midpoints between adjacent cluster centroids, meaning they are entirely derived from the data's distribution.

Strengths

- Captures natural data structure
- No assumptions on bin width or count
- Flexible and data-driven approach
- Boundaries reflect actual data density patterns

Weaknesses

- Computationally more expensive
- Results depend on k-means initialization
- Harder to interpret than fixed-boundary methods
- Less deterministic due to k-means

❏ Hartigan & Wong (1979) introduced the foundational "Algorithm AS 136: A K-Means Clustering Algorithm" in the Journal of the Royal Statistical Society.

Clustering-Based Discretization

```
library(daltoolbox)

iris <- datasets::iris

# Discretização por clustering (k=2)
obj <- smoothing_cluster(n = 2)
obj <- fit(obj, iris$Sepal.Length)
sl.bi <- transform(obj, iris$Sepal.Length)
print(table(sl.bi))
# 5.199 6.580
# 80 70
obj$interval
# [1] 4.300000 5.889375 7.900000
```

```
# Entropia condicional
entro <- evaluate(obj,
  as.factor(names(sl.bi)), iris$Species)
print(entro$entropy) # [1] 1.097573

# Otimizando número de bins (1:20)
opt_obj <- smoothing_cluster(n=1:20)
obj <- fit(opt_obj, iris$Sepal.Length)
obj$sn # [1] 8
```

Step by step

While `smoothing_freq` balances counts via quantiles, `smoothing_cluster` goes further: instead of applying fixed rules, it **lets the data define its own cut points** through 1D k-means clustering.

- Bins defined by clustering (1D k-means) instead of fixed interval rules
- Boundaries emerge from the data: cut point at 5.889 (vs. 5.8 by frequency)
- Same entropy (1.097) and same optimal number of bins (8) as the frequency method
- More flexible: cluster centers guide the natural cut points of the data

📌 **Synthesis of the three methods:** Use `smoothing_inter` when interpretability of equal-width intervals is a priority; prefer `smoothing_freq` for skewed distributions, as quantiles avoid empty bins; choose `smoothing_cluster` when you want boundaries to emerge naturally from the data, without assumptions about width or count. All three are available in the **daltoolbox** package and follow the same `fit / transform / evaluate` interface, making it easy to swap and compare approaches.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

